StockPulse — Schema Document

Database choice: Firebase (Cloud Firestore + file storage)
App language: TypeScript / JavaScript (Next.js)
This document does not describe a step-by-step migration script. It describes **what the database is for**, **why Firebase**, **data models**, **access control**, and **storage / Blob**.

---

1. What the database is for

The database stores everything that belongs to a business:

- Business profile and settings
- Products and quantities in **warehouse / shop / online**
- Sales and price history
- Owners and staff
- Alerts
- Backup history metadata
- Login-related records (OTP / sessions) as designed on the server

Each business is a **tenant**. Data from business A must never appear for business B.

---

2. Why we choose Firebase

Reason	Explanation
Document model	Inventory fits collections and documents (business → products, people, sales).
Managed service	No need to run your own database server for day-to-day use.
Scales with users	Grows as more businesses and products are added.
Storage companion	Firebase Storage (or blob-compatible storage) holds backup files and photos.
Works with our stack	Next.js API routes on Vercel can use the Firebase Admin SDK for trusted writes.
Security model	Server role checks plus optional Firestore Security Rules.

Login remains WhatsApp OTP through our APIs. Firebase is the **data** platform; we are not requiring Gmail login for users.

---

3. How the database is made (structure)

Firebase **Cloud Firestore** is a **NoSQL document database**.

- **Collection** = a group of similar records (e.g. `products`).
- **Document** = one record with fields (e.g. one product).
- **Subcollection** (optional) = nested lists under a business (e.g. `businesses/{id}/products`).

Logical model (names may be adjusted in implementation, meaning stays the same):

```
businesses/{businessId}
  ├── products/{productId}
  ├── owners/{ownerId}
  ├── staff/{staffId}
  ├── sales/{saleId}
  ├── alerts/{alertId}
  ├── backupHistory/{backupId}
  └── (auth helpers as designed: otp / sessions)
```

---

4. Data models

4.1 Business

Field	Meaning
id	Tenant key
name	Business name
language	UI language
priceThreshold	% for price alerts
connectedStores	Shopify / store links count
lastBackup	Last successful backup time
isSample	Sample-data flag

4.2 Product

Field	Meaning
name, category, variant	Identity
warehouseQty, shopQty, onlineQty	**Three separate locations**
purchasePrice	Current purchase price

restockThreshold	Low-stock line
priceHistory	Month + price points
photo	Optional image URL
shopifyVariantId	Optional Shopify link

Rule: total quantity = warehouse + shop + online (computed).

4.3 Sale

Field	Meaning
productId	Which product
location	warehouse / shop / online
quantity, unitPrice	Deduction
recordedAt	When

4.4 Owner

Field	Meaning
name, phone	WhatsApp identity
access	equal / co
isPrimary	Primary owner

4.5 Staff

Field	Meaning
name, phone	Required
permission	add / view

4.6 Alert

Field	Meaning
kind, title, body	Message
productId	Optional link
seen	Read or not
createdAt	Time

4.7 Backup history (metadata in DB)

Field	Meaning
stampedAt	Exact date and time
status	success / failed
storagePath	Path or URL of the file
error	Message if failed

---

5. Access control

Access is decided by **role** after WhatsApp login.

Role	Access
equal_owner	Full access including managing owners and reset
co_owner	Owner features with limited owner-management
staff_add	Add products + view items; no dashboard/settings/prices
staff_view	View items only

Enforcement:

1. UI **Gate** blocks wrong pages.
2. API checks role before write/read of sensitive data.
3. Firestore rules (when used) must not allow one business to read another.

Owner-only examples: dashboard, settings, staff directory, price pages, system status, sending owner reports.

---

6. Storage / Blob — what it means

Blob means a **binary file** stored outside the main document database: a backup dump, a photo, a PDF export, etc.

Question	Answer for StockPulse
What is stored?	Daily backups (date+time stamped), optional product photos
Where?	Object storage — Firebase Storage and/or Vercel Blob-style storage
What is in Firestore?	Only **metadata** (when, success/fail, path, error)

Why not put the whole backup inside Firestore?	Files are large; document DBs are for structured fields, not big binary dumps
How much space?	Depends on catalog size and backup frequency; each backup is roughly the size of that business's exported data. History keeps multiple stamped copies, so space grows with retention

Owners see **Backup History** in Settings. Failed backups keep an error note for logging and checks.

---

7. Language and tooling note

- Application code: **TypeScript / JavaScript**.
- Database service: **Firebase (Firestore + Storage)**.
- Hosting that serves the app and APIs: **Vercel**.
- Secrets (service account JSON, API keys): **never commit to Git**.

---

8. Related documents

1. Master Requirements Document
2. PRD
3. TRD — overall technical requirements
4. Flow diagrams — login and stock/backup flows
5. Brand kit — colors and typography in use
6. Testing — how we verify this model every day